Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа №3

По дисциплине «Модели решения задач в интеллектуальных системах»

Тема: «Бинарная классификация с использованием кросс-энтропии (логистическая регрессия)»

Выполнил:

Студент 3 курса

Группы ИИ-26

Петручик Д.А.

Проверила:

Андренко К.В.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE). Провести сравнительный анализ скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Ход работы:

Задачи лабораторной работы:

1. Модифицировать программу из ЛР №1:
   - Заменить пороговую функцию активации на сигмоидную.
   - Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
   - Реализовать вычисление градиента ŷ-exi для обновления весов.
2. Реализовать обучение с фиксированным шагом (взять значение α, показавшее наилучший результат в ЛР №1).
Запоминать суммарную ошибку Esp на каждой эпохе p.
3. Реализовать обучение с адаптивным шагом:
   - На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
   - Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
   - Сохранять значения Esp для построения графика.
4. Провести сравнительный анализ:
   - Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
   - MSE + фиксированный шаг (данные из ЛР №1);
   - MSE + адаптивный шаг (данные из ЛР №2);
   - BCE + фиксированный шаг;
   - BCE + адаптивный шаг.
   - Зафиксировать число эпох, необходимое для достижения заданной точности Es ≤Ee (взять Ee из ЛР №2, например 0.01).
5. Визуализировать результаты:
   - Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
   - Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
   - Для сравнения также показать линию из ЛР №1 (если она отличается). Описать сравнение двух линий.
6. Реализовать режим функционирования:
   - Пользователь вводит координаты x1,x2.
   - Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
   - Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).
7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.
8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

10.

x_1	x_2	e
6	6	0
-6	6	0
6	-6	1
-6	-6	0

x_1, x_2 - входные данные сети, e - эталонные значения

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

X = np.array([
    [6, 6],
        [-6, 6],
        [6, -6],
        [-6, -6]
], dtype = float)

E = np.array([0, 0, 1, 0], dtype = float)

T_MSE = np.where(E == 0, -1, 1)

alpha = 0.01
epochs = 500
Ee = 0.01

def net(x, w, b) :
    return np.dot(w, x) + b


        def step(u) :
        return 1 if u >= 0 else 0


        def sigmoid(z) :
        z = np.clip(z, -500, 500)
        return 1 / (1 + np.exp(-z))


        def bce_loss(y, yhat) :
        yhat = np.clip(yhat, 1e-12, 1 - 1e-12)
        return -(y * np.log(yhat) + (1 - y) * np.log(1 - yhat))


        def train_mse_fixed(X, T) :

        w = np.zeros(2)
        b = 0
        history = []

        for epoch in range(epochs) :

            Es = 0
```

```
        for x, t in zip(X, T) :

            y = net(x, w, b)

            err = t - y

            Es += err * *2

            w += alpha * err * x
            b += alpha * err

            history.append(Es)

            if Es <= Ee:
break

    return w, b, history

    def train_mse_adaptive(X, T) :

    w = np.zeros(2)
    b = 0
    history = []

    for epoch in range(epochs) :

        Es = 0

        for x, t in zip(X, T) :

            y = net(x, w, b)

            err = t - y

            Es += err * *2

            alpha_t = 1 / (np.dot(x, x) + 1)

            w += alpha_t * err * x
            b += alpha_t * err

            history.append(Es)

            if Es <= Ee:
break

    return w, b, history

    def train_bce_fixed(X, E) :

    w = np.zeros(2)
    b = 0
    history = []

    for epoch in range(epochs) :

        Es = 0

        for x, e in zip(X, E) :

            z = net(x, w, b)

            yhat = sigmoid(z)

            err = yhat - e
```

```
            Es += bce_loss(e, yhat)

            w -= alpha * err * x
            b -= alpha * err

            history.append(Es)

            if Es <= Ee:
break

    return w, b, history

    def train_bce_adaptive(X, E) :

    w = np.zeros(2)
    b = 0
    history = []

    for epoch in range(epochs) :

        Es = 0

        for x, e in zip(X, E) :

            z = net(x, w, b)

            yhat = sigmoid(z)

            err = yhat - e

            Es += bce_loss(e, yhat)

            alpha_t = 1 / (np.dot(x, x) + 1)

            w -= alpha_t * err * x
            b -= alpha_t * err

            history.append(Es)

            if Es <= Ee:
break

    return w, b, history

    w1, b1, h1 = train_mse_fixed(X, T_MSE)
    w2, b2, h2 = train_mse_adaptive(X, T_MSE)
    w3, b3, h3 = train_bce_fixed(X, E)
    w4, b4, h4 = train_bce_adaptive(X, E)

    plt.figure(figsize = (9, 5))

    plt.plot(h1, label = "MSE fixed")
    plt.plot(h2, label = "MSE adaptive")
    plt.plot(h3, label = "BCE fixed")
    plt.plot(h4, label = "BCE adaptive")

    plt.yscale("log")
    plt.xlabel("Epoch")
    plt.ylabel("Es")
    plt.title("Сравнение сходимости")
    plt.legend()
    plt.grid()

    plt.show()
```

```
        plt.figure(figsize = (8, 6))

        for i in range(len(X)) :

            if E[i] == 0 :
                plt.scatter(X[i, 0], X[i, 1], color = "blue", s = 120, label =
"class 0" if i == 0 else "")
            else :
                plt.scatter(X[i, 0], X[i, 1], color = "red", s = 120, label = "class
1")

                x_line = np.linspace(-10, 10, 100)

                y_mse = -(w1[0] * x_line + b1) / w1[1]
                y_bce = -(w4[0] * x_line + b4) / w4[1]

                plt.plot(x_line, y_mse, '--', label = "MSE line")
                plt.plot(x_line, y_bce, label = "BCE line")

                plt.xlabel("x1")
                plt.ylabel("x2")
                plt.title("Разделяющие линии")
                plt.legend()
                plt.grid()

                plt.show()

                print("\nРежим тестирования")
                print("Введите x1 x2 или q")

                while True:

    s = input(">> ")

        if s == "q" :
            break

            try :

            x1, x2 = map(float, s.replace(",", " ").split())

            z = net(np.array([x1, x2]), w4, b4)

            prob = sigmoid(z)

            cls = 1 if prob >= 0.5 else 0

            print("Вероятность =", round(prob, 4))
            print("Класс =", cls)

            except:
    print("Ошибка ввода")
```

Результат программы:

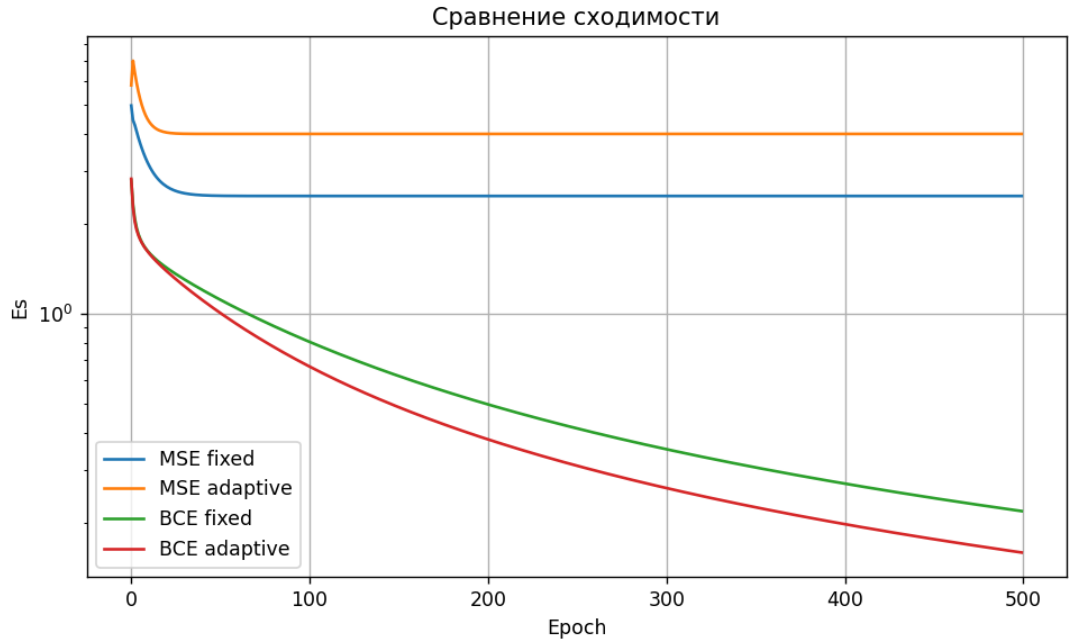
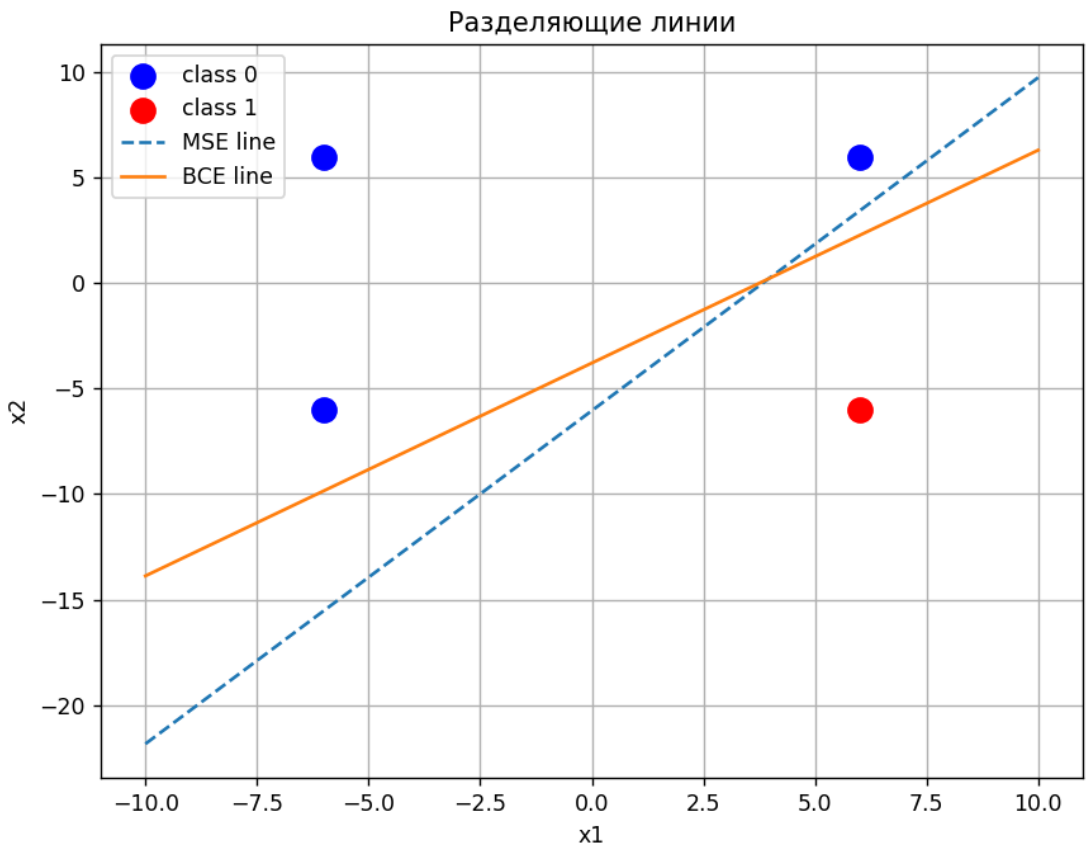


График с разделяющей:



```
Введите x1 x2 или q
>> 6,6
Вероятность = 0.0747
Класс = 0
>> 6,-6
Вероятность = 0.9962
Класс = 1
>> -6,6
Вероятность = 0.0
Класс = 0
>> -6,-6
Вероятность = 0.0698
Класс = 0
>>
```

Вывод: Проделав данную лабораторную работу я изучил принципы бинарной классификации с вероятностной точки зрения. Реализовал однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE). Провел сравнительный анализ скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).